

Debugging & Testing

Writing code that runs is the easy half. Proving it does the right thing — and tracking down why it doesn't — is the skill examiners reward. This module covers test tables, constructing test data, and the debugging techniques every VCE programmer is expected to name and use.

What this key knowledge covers

This summary distils everything you need for **KK12 — Debugging & Testing** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Testing finds the bug. Debugging removes it.

These two words get swapped all the time, and examiners love punishing it. They are different jobs done in a different order with different tools.

□ Testing

Running the solution with chosen **test data** and comparing the **actual result** against the **expected result** you worked out beforehand. Testing *detects* whether a fault exists — it doesn't fix anything.

Main tool: the **test table**.

□ Debugging

The process of **locating and correcting** the cause of a fault that testing has revealed. Debugging *diagnoses and repairs*.

Main tools: **breakpoints, debugging output statements, and desk checking / trace tables**.

Testing is the *thermometer and the blood test* — measurements that tell you something is wrong. **Debugging** is the *diagnosis and the treatment* — finding the cause and fixing it. You can take someone's temperature all day; that never cures them. Test data is the symptom you provoke on purpose so the illness shows up.

The three kinds of error

Every fault you'll ever test or debug falls into one of three families. Naming the family correctly is half the battle, because each one is caught a different way.

Syntax error

You broke the **grammar rules** of the language — a missing colon, an unclosed bracket, a misspelled keyword. The program **won't run at all**; the interpreter/compiler refuses.

```
prnit("hi") · if x > 5 (no colon)
```

Caught by: the editor / interpreter, instantly.

Logic error

The code runs happily and produces an answer — just the **wrong answer**. The grammar is fine; the **thinking** is faulty. The sneakiest of the three.

Using `<` where you needed `<=` ; adding when you meant to multiply.

Caught by: a **test table** — nothing else flags it.

Run-time error

The program starts fine, then **crashes part-way** when it hits something impossible — dividing by zero, reading a file that isn't there, an index past the end of an array.

```
total / count when count = 0
```

Caught by: an error/exception while running.

Error triage

Drag each fault into the bin you think it belongs in. The dock keeps your score.

Placed correctly: **0 / 8**. A program that *runs* never has a syntax error.

The Boundary Bug Hunter

Here is Zoe's ParkPass pricing module on a 3D **age axis**. Drag the slider to set a customer's age and watch the execution token fall into a price band. The **boundaries** between bands — the exact ages 4, 16 and 65 — are where logic bugs hide. Flip between her buggy code and the fixed code and watch what happens right at age 15.

Anatomy of a test table

A **test table** (testing table) is the structured record of testing. The non-negotiable idea is the **expected result is worked out by hand before you run the code** — with a calculator, pen and paper, your brain — and only then do you record what the program actually produced.

Must-have columns

Test number · what it checks · **test data** · **expected result** · **actual result** · **pass/fail**. Some teachers add a "corrective action" column for what you'll change when it fails.

The golden rule

Expected $\neq$ Actual $\Rightarrow$ **FAIL** $\Rightarrow$ now you debug. Expected = Actual $\Rightarrow$ PASS for *that input only*. One pass never proves the whole module works.

Constructing test data

Good testing lives or dies on the **data you choose to feed in**. Random numbers won't do it. You deliberately construct three types so that, together, they exercise every path through the code and every edge of every range.

NORMAL

Typical, valid, "middle of the range" values the program should obviously accept. Proves the everyday case works.

BOUNDARY

Values **right on the edge** of a range and one either side — the cut-offs where a wrong $<$ vs $\leq$ shows up. The single highest-value test data you can write.

ParkPass: 3, 4 · 15, 16 · 64, 65.

ABNORMAL

Invalid / unexpected input the program should **reject or handle** — negatives, letters, blanks. Proves validation holds and the program won't crash.

ParkPass: age = -5, age = "old", blank.

Before opening a bridge, engineers don't just drive one normal car across (*normal data*). They park a fully loaded truck right at the rated weight limit (*boundary data*), and they imagine a flood or an earthquake (*abnormal data*). The boundary test — exactly at the limit — is the one that reveals whether the safety margin was calculated with $<$ or $\leq$. Same logic in code.

Live test-table builder

Add test rows for the **buggy** ParkPass module below. Type an age, predict the expected price, then press *Run* — the real buggy function executes and fills the actual result, marking pass/fail for you. Your job: find an age where it **fails**. (Hint: the buggy version uses age < 15 for the junior cut-off instead of age < 16 .)

Debugging techniques: stepping the code

Testing told us *that* ParkPass is wrong. Debugging finds *where*. The two techniques VCE expects you to name are **breakpoints** and **debugging output statements** — plus the old-school **desk check / trace**. Step the execution token line by line and watch the variables update in the watch panel.

● Breakpoint

A marker that **pauses execution** at a chosen line so you can inspect the current value of every variable *at that moment*. You watch age = 15 reach line 4, see age < 15 is false, and realise it wrongly skips to the adult band. No code is changed to use one.

□ Debugging output statement

A temporary print() line you **insert into the code** to display a variable's value as the program runs — e.g. print("age=", age, "price=", price). Cheap, works anywhere, but you must remember to **remove them** before release.

The pay-calculator worked example

VCAA's own teaching activity for this dot point asks students to test a **pay calculator**: enter hours worked and hourly rate; gross pay is calculated; **12% is removed for superannuation** and **30% for tax**; net pay is shown. You compute the expected result with pen and calculator, run the program, and compare. Here it is as a finished test table.

Worked by hand for **10 hours @ \$20/hr**: gross = $10 \times 20 = \$200$; super = $12\% = \$24$; tax = $30\% = \$60$; net = $200 - 24 - 60 = \$116$.

1 • Mixing up the two words

Giving a debugging tool (breakpoint) when the question asked for a testing technique (test table), or vice versa. Read the command word.

2 • No boundary data

A test table of three "normal" rows that all pass. It demonstrates nothing. Always include boundary and abnormal rows.

3 • Forgetting "expected ≠ actual"

Describing a test table without naming both an **expected** and an **actual** column. The comparison *is* the technique.

4 • "I'd just run it and see"

Expected results must be calculated **by hand first**. If you read the answer off the program, you're testing the program against itself.

5 • Naming an error type wrong

Calling a wrong-answer-but-runs fault a "syntax error". A program that runs has no syntax error — it's a **logic error**.

6 • One example, no link

Higher-mark questions want the technique *plus* why it suits this scenario. Always close the loop back to the case study.

P-E-L writing trainer

Short-answer marks come from structure, not length. Make a **Point**, give **Evidence** from the scenario, then **Link** it to the consequence. Tab through a model, then try your own — keyword hits light up.

Your turn

Prompt: *Zoe's ParkPass module runs but charges 16-year-olds the wrong price. Explain one testing technique she could use to detect this fault.*

VCE-style exam questions

Five questions, **20 marks** total. Attempt each in your head or on paper, reveal the weak vs strong responses, then self-mark — the dock tallies your score.

Cheat sheet

- **Testing** = detect a fault (test table)
- **Debugging** = locate + fix it
- Test → find → debug → re-test
- Test # · what it checks
- **Test data** (input)
- **Expected** (worked by hand first)
- **Actual** · **Pass/Fail**
- NORMAL typical valid value
- BOUNDARY on the edge ± 1
- ABNORMAL invalid / rejected
- **Syntax** — won't run (grammar)
- **Logic** — runs, wrong answer
- **Run-time** — crashes mid-run
- **Breakpoint** — pause & inspect variables
- **Output statement** — temp print() of a value
- **Desk check / trace table** — hand-execute on paper
- Always include a **boundary** row
- Name **expected AND actual**
- **Link** the fault to the scenario
- Tie abnormal tests to **validation**

That's KK12 — the final dot point of the Programming outcome. □

Test deliberately, debug methodically, and always write the expected result first. More modules at digitalvector.com.au

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Practice question

► Reveal sample responses &

✗ A weak answer

er response Testing and debugging are both about fixing errors in a program to make it work properly. Treats them as the same thing — no distinction, which is exactly what the question asks for.

✓ A strong answer

er response Testing is the process of **detecting** whether a fault exists by running the solution with test data and comparing the actual result to the expected result. Debugging is the separate process of **locating and correcting** the cause of a fault that testing has revealed. Two clearly different jobs: detect vs. locate-and-fix. That contrast earns both marks.

How the marks are awarded

- 1 mark: testing detects/identifies faults (e.g. via expected vs actual).
- 1 mark: debugging locates and corrects the cause of a fault.